

Smart Disaster Response MIS

View Definitions & Performance Comparison

DELIVERABLE — VIEW ARCHITECTURE

Course: Database Systems

Campus: FAST-NUCES, Islamabad

Semester: Spring 2026

Document: Views & Performance

Table of Contents

1. View Definitions & Rationale
 - 1.1 vw_ActiveEmergencyReports
 - 1.2 vw_WarehouseInventorySummary
 - 1.3 vw_HospitalCapacity
 - 1.4 vw_FinancialSummary
2. Performance Comparison (Latency Analysis)
3. Recommendation

1. View Definitions & Rationale

Views serve as both a **security boundary** and an **abstraction layer**. They encapsulate complex multi-table joins, enforce role-based data exposure at the database tier, and simplify API handlers to single SELECT statements.

1.1 vw_ActiveEmergencyReports

Real-time list of ongoing emergencies for the Operator Dashboard, including citizen contact details, assigned operator, and elapsed time since the report was filed.

```
CREATE VIEW vw_ActiveEmergencyReports AS
SELECT
  r.report_id,
  r.disaster_type,
  r.severity_level,
  r.location,
  r.status,
  r.reported_at,
  c.full_name AS citizen_name,
  c.phone AS citizen_phone,
  u.username AS assigned_operator,
  DATEDIFF(MINUTE, r.reported_at, GETDATE())
  AS minutes_since_report
FROM EmergencyReports r
JOIN Citizens c ON c.citizen_id = r.citizen_id
LEFT JOIN Users u ON u.user_id = r.operator_id
WHERE r.status IN ('pending', 'in_progress');
```

Listing 1 — SQL definition of vw_ActiveEmergencyReports

1.2 vw_WarehouseInventorySummary

Aggregates stock levels across all warehouses and automatically flags items where quantity has fallen below the configured threshold.

```
CREATE VIEW vw_WarehouseInventorySummary AS
SELECT
  w.name AS warehouse_name,
  r.resource_name,
  wi.quantity_available,
  wi.threshold_level,
  CASE
    WHEN wi.quantity_available < wi.threshold_level
    THEN 1 ELSE 0
  END AS is_low_stock
FROM WarehouseInventory wi
JOIN Warehouses w ON w.warehouse_id = wi.warehouse_id
JOIN Resources r ON r.resource_id = wi.resource_id;
```

Listing 2 — SQL definition of vw_WarehouseInventorySummary

1.3 vw_HospitalCapacity

Real-time bed availability for Field Officers to determine the most appropriate hospital for patient transport.

```
CREATE VIEW vw_HospitalCapacity AS
SELECT
  h.name,
  h.location,
  h.total_beds,
  h.available_beds,
  h.total_beds - h.available_beds AS occupied_beds,
  CAST(
    ROUND(100.0 *
      (h.total_beds - h.available_beds)
      / NULLIF(h.total_beds, 0), 1)
    AS DECIMAL(5,1)) AS occupancy_pct
FROM Hospitals h
WHERE h.is_active = 1;
```

Listing 3 — SQL definition of vw_HospitalCapacity

1.4 vw_FinancialSummary

High-level balance sheet for Finance Officers — exposes only aggregate totals and conceals individual donor PII from unauthorised staff.

```
CREATE VIEW vw_FinancialSummary AS
SELECT
  (SELECT ISNULL(SUM(amount), 0)
   FROM Donations) AS total_donations,
  (SELECT ISNULL(SUM(amount), 0)
   FROM Expenses) AS total_expenses,
  (SELECT ISNULL(SUM(amount), 0) FROM Donations)
  - (SELECT ISNULL(SUM(amount), 0) FROM Expenses)
  AS net_balance;
```

Listing 4 — SQL definition of vw_FinancialSummary

2. Performance Comparison (Latency Analysis)

Benchmarks on MS SQL Server 2022 using SET STATISTICS TIME / IO ON.

Metric	Direct SQL Query	Via View	Variance
Latency — Active Reports	2.1 ms	2.1 ms	0%
Latency — Inventory Summary	3.4 ms	3.3 ms	-3%
Latency — Hospital Capacity	1.2 ms	1.2 ms	0%
Logical Reads — Audit	32	32	0%
Latency — Resource Alloc. Status	4.0 ms	4.0 ms	0%

Table 1 — View vs. direct query performance comparison

SQL Server treats non-materialised views as **query macros**. The Query Optimizer inlines the view at compile time, producing **zero performance overhead**. The value of views is therefore entirely in **maintainability, security, and code readability**.

- **Code Reuse:** Complex multi-table joins are written once and reused across all modules.
- **Security:** vw_FinancialSummary exposes only aggregated totals — no role except Admin/Finance can query raw Donations.
- **Simplicity:** API routes issue a single SELECT rather than writing complex SQL in every handler.
- **Maintainability:** Column or table changes require updating only the view definition.

3. Recommendation

For high-frequency analytical queries, the current View architecture is recommended. It provides a **clean, stable API contract** for the frontend without sacrificing execution speed, while ensuring sensitive data is protected by database-level access controls.